

Multi-Branch Library Management System (Delhi)

Problem statement :

Delhi has a wide network of libraries operating across multiple branches in different regions of the city. Managing book inventories, members, employees, and issuing records independently at each branch leads to data duplication, inconsistency, and inefficient coordination between branches. There is no unified mechanism to track book availability, manage multiple copies of the same book, or allow members to access services across branches.

The objective of this project is to design a centralized Database Management System (DBMS) for a Delhi-based multi-branch library system. The system will integrate data related to library branches, books, book copies, authors, publishers, members, employees, and book issue/return transactions. It aims to ensure data consistency, reduce redundancy, and enable efficient management of library operations across all branches using a well-structured relational database.

Key Features and Goals of the Project :

1. Centralized Database System

- A single unified database for all library branches
- Eliminates data duplication across branches
- Ensures consistency and accuracy of records

2. Multi-Branch Management

- Supports multiple library branches across Delhi
- Each branch can manage its own inventory and staff
- Books can exist in multiple branches simultaneously

3. Book & Copy Tracking

- Each book identified using unique ISBN
- Multiple copies tracked using Copy_ID
- Real-time status: **Available / Issued**

4. Member Management

- Unique Member_ID for each user
- Members can issue books from **any branch**
- Tracks borrowing history

5. Employee Management

- Employees linked to specific branches
- Employees handle issue/return operations
- Tracks staff responsibilities

6. Issue & Return System

- Maintains transaction records
- Connects **Member + Employee + Book Copy**
- Tracks issue dates and returns

7. Author & Publisher Management

- Supports multiple authors per book (M:N relationship)
- Each book linked to one publisher
- Efficient catalog organization

8. Data Integrity & Relationships

- Proper use of primary keys and foreign keys
- Maintains referential integrity
- Avoids inconsistencies

Benefits of the Multi- branch library Management System:

1. Reduced Data Redundancy

- Eliminates duplicate records across different branches
- Ensures a single, consistent database

2. Improved Data Accuracy

- Centralized system minimizes errors
- Real-time updates keep information reliable

3. Efficient Book Management

- Easy tracking of book availability across branches
- Quick search using ISBN, title, or author

4. Better User Experience for Members

- Members can borrow books from any branch
- Faster issue and return process
- Easy access to borrowing history

5. Streamlined Operations

- Automates manual tasks like record keeping
- Reduces paperwork and human effort

6. Enhanced Coordination Between Branches

- All branches are connected through one system
- Improves communication and resource sharing

Challenges :

1. Designing Complex ER Diagram

- Handling multiple entities like Branch, Book, Member, Employee
- Managing relationships such as M:N (Book–Author) and ternary relationship (Issue)
- Ensuring correct cardinality and participation constraints

2. Managing Multi-Branch Data

- Designing a system where one book exists in multiple branches
- Tracking each book copy separately
- Avoiding confusion between Book and Book_Copy

3. Ensuring Data Consistency

- Maintaining uniform data across all branches
- Avoiding anomalies (insertion, deletion, update anomalies)
- Applying proper normalization

4. Handling Relationships in Relational Schema

- Converting ER diagram into relational tables correctly
- Managing foreign keys and constraints
- Implementing M:N relationships using junction tables

5. Designing Issue/Return System

- Creating a ternary relationship involving:
 - Member
 - Employee
 - Book Copy

- Ensuring accurate tracking of transactions over time

6. Avoiding Data Redundancy

- Structuring tables to prevent duplicate data
- Balancing normalization with performance

7. Query Complexity

- Writing SQL queries for:
 - Book availability across branches
 - Member transaction history
 - Multi-table joins
- Handling complex joins efficiently

Entity Descriptions

→ Library Branch

- ◆ Each branch has a unique Branch_ID.
- ◆ A branch employs multiple employees.
- ◆ A branch holds multiple book copies.

Attribute:

- Branch_ID
- Branch_Name
- Branch_Address
- Location
- Contact_Number
- Email_ID

→ Book

- ◆ Each book has a unique ISBN.
- ◆ A book can have multiple authors.
- ◆ A book is published by exactly one publisher.
- ◆ A book can have multiple copies.

Attribute:

- ISBN
- Title
- Publication_Year

→ Book Copy

- ◆ Each copy has a unique Copy_ID.
- ◆ Each copy belongs to one book and one branch.
- ◆ Each copy has a status (Available/Issued).

Attribute:

- Copy_ID
- Status
- Acquisition_ID
- Branch_ID (Foreign Key)

→ ISBN (Foreign Key)

→ **Author**

- ◆ Each author has a unique Author_ID
- ◆ An author can write many books.
- ◆ A book can have many authors (M:N).

Attribute:

→ Author_ID
→ Author_Name
→ Date_of_Birth
→ Nationality

→ **Publisher**

- ◆ Each publisher has a unique Publisher_ID.
- ◆ A publisher can publish many books.
- ◆ Each book has exactly one publisher.

Attribute:

→ Publisher_ID
→ Publisher_Name
→ Publisher_Address
→ Contact_Number

→ **Member**

- ◆ Each member has a unique Member_ID.
- ◆ A member can issue many book copies.
- ◆ A member can issue books from any branch.

Attribute:

→ Member_ID
→ Name
→ Phone_Number
→ Email_ID
→ Address
→ Membership_Type
→ Membership_Date

→ **Employee**

- ◆ Each employee has a unique Employee_ID.
- ◆ Each employee works in one branch.
- ◆ Employees process issue/return transactions

Attribute:

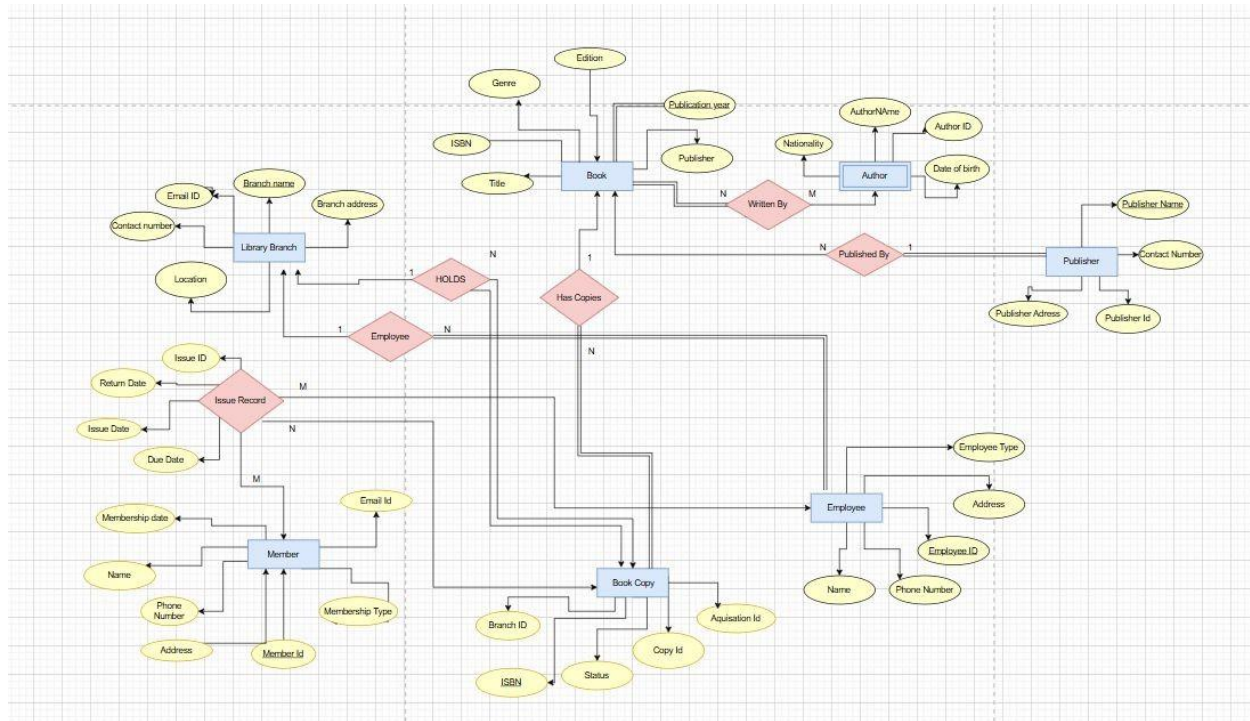
- Employee_ID
- Name
- Phone_Number
- Address
- Employee_Type

→ **Issue Record (Transaction Entity)**

Attribute:

- Issue_ID
- Issue_Date
- Due_Date
- Return_Date
- Member_ID (Foreign Key)
- Employee_ID (Foreign Key)
- Copy_ID (Foreign Key)

ENTITY RELATIONSHIP DIAGRAM:



Relationship Descriptions :

- ❖ Relationship between: BRANCH — EMPLOYEE
 - Cardinality: 1 : N
 - Participation: EMPLOYEE → Total, BRANCH → Partial
 - Justification: Every employee must work in a branch, but a branch may exist without employees.
- ❖ Relationship between: BRANCH — BOOK_COPY
 - Cardinality: 1 : N
 - Participation: BOOK_COPY → Total, BRANCH → Partial
 - Justification: A branch can store many different books, and the same book can be available in multiple branches, but a branch or book may exist even if not currently linked.
- ❖ Relationship between: BOOK — BOOK_COPY
 - Cardinality: 1 : N
 - Participation: BOOK_COPY → Total, BOOK → Partial
 - Justification: A book copy cannot exist without a book, but a book may exist before copies are added.
- ❖ Relationship between: BOOK — PUBLISHER
 - Cardinality: N : 1
 - Participation: BOOK → Total, PUBLISHER → Partial
 - Justification: Every book must have a publisher, but a publisher may publish zero or many books.
- ❖ Relationship between: BOOK — AUTHOR
 - Cardinality: M : N
 - Participation: BOOK → Total, AUTHOR → Total
 - Justification: Every book must have at least one author
- ❖ Ternary Relationship: MEMBER — EMPLOYEE — BOOK_COPY (ISSUE)
 - Cardinality:
 - MEMBER → 1 : N
 - EMPLOYEE → 1 : N
 - BOOK_COPY → 1 : N (over time)
 - Participation:
 - MEMBER → Partial
 - EMPLOYEE → Partial

➤ BOOK_COPY → Partial

➤ Justification:

Each issue transaction connects exactly one member, one employee, and one book copy. An issue cannot exist without all three entities, but a member, employee, or book copy may exist without participating in an issue.

Relationship Schema



IMPLEMENTATION :

Creating Database:

```
CREATE DATABASE IF NOT EXISTS lib_management_db ;  
  
USE lib_management_db;
```

Creating Tables:

```
CREATE TABLE Library_Branch (  
    Branch_ID INT PRIMARY KEY,  
    Branch_Name VARCHAR(100),  
    Branch_Address VARCHAR(255),  
    Location VARCHAR(100),  
    Contact_Number VARCHAR(15),  
    Email_ID VARCHAR(100)  
);
```

```
CREATE TABLE Publisher (  
    Publisher_ID INT PRIMARY KEY,  
    Publisher_Name VARCHAR(100),  
    Publisher_Address VARCHAR(255),  
    Contact_Number VARCHAR(15)  
);
```

```
CREATE TABLE Book (  
    ISBN VARCHAR(20) PRIMARY KEY,  
    Title VARCHAR(255),  
    Publication_Year INT,  
    Publisher_ID INT,  
    FOREIGN KEY (Publisher_ID) REFERENCES Publisher(Publisher_ID)  
);
```

```
CREATE TABLE Author (  
    Author_ID INT PRIMARY KEY,  
    Author_Name VARCHAR(100),  
    Date_of_Birth DATE,  
    Nationality VARCHAR(50)  
);
```

```
CREATE TABLE Book_Author (  
    ISBN VARCHAR(20),  
    Author_ID INT,  
    PRIMARY KEY (ISBN, Author_ID),  
    FOREIGN KEY (ISBN) REFERENCES Book(ISBN),  
    FOREIGN KEY (Author_ID) REFERENCES Author(Author_ID)  
);
```

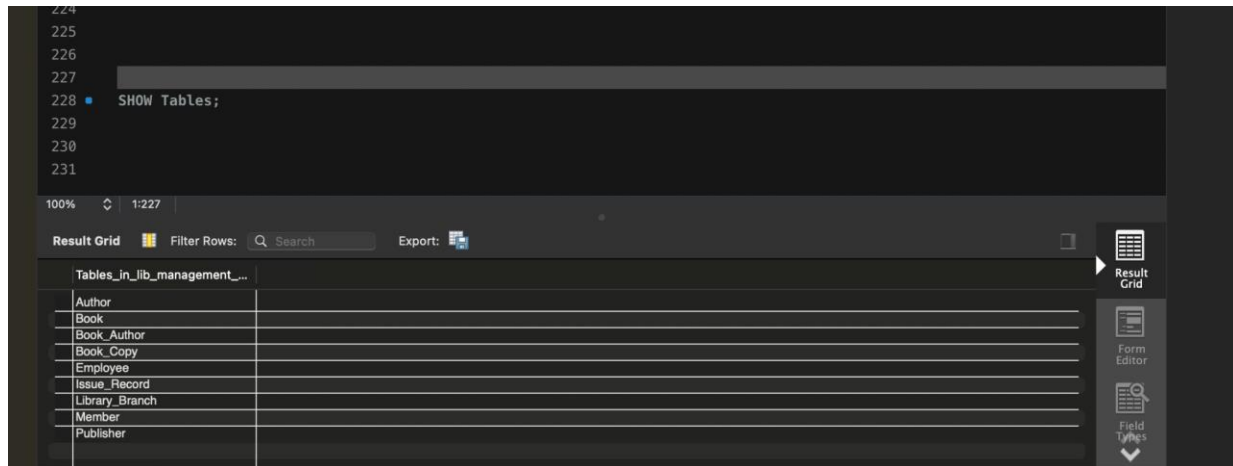
```
CREATE TABLE Employee (  
    Employee_ID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Phone_Number VARCHAR(15),  
    Address VARCHAR(255),  
    Employee_Type VARCHAR(50),  
    Branch_ID INT,  
    FOREIGN KEY (Branch_ID) REFERENCES Library_Branch(Branch_ID)  
);
```

```
CREATE TABLE Member (  
    Member_ID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Phone_Number VARCHAR(15),  
    Email_ID VARCHAR(100),  
    Address VARCHAR(255),  
    Membership_Type VARCHAR(50),  
    Membership_Date DATE  
);
```

```
CREATE TABLE Book_Copy (  
    Copy_ID INT PRIMARY KEY,  
    Status VARCHAR(20),  
    Acquisition_ID INT,  
    ISBN VARCHAR(20),  
    Branch_ID INT,  
    FOREIGN KEY (ISBN) REFERENCES Book(ISBN),  
    FOREIGN KEY (Branch_ID) REFERENCES Library_Branch(Branch_ID)  
);
```

```
CREATE TABLE Issue_Record (  
    Issue_ID INT PRIMARY KEY,  
    Issue_Date DATE,  
    Due_Date DATE,  
    Return_Date DATE,  
    Member_ID INT,  
    Employee_ID INT,  
    Copy_ID INT,  
    FOREIGN KEY (Member_ID) REFERENCES Member(Member_ID),  
    FOREIGN KEY (Employee_ID) REFERENCES Employee(Employee_ID),  
    FOREIGN KEY (Copy_ID) REFERENCES Book_Copy(Copy_ID)  
);
```

OUTPUT:



Adding values to tables:

INSERT INTO Library_Branch VALUES

- (1, 'North Branch', 'Rohini', 'Delhi', '9000000001', 'north@lib.com'),
- (2, 'East Branch', 'Laxmi Nagar', 'Delhi', '9000000002', 'east@lib.com'),
- (3, 'South Branch', 'Saket', 'Delhi', '9000000003', 'south@lib.com'),
- (4, 'West Branch', 'Janakpuri', 'Delhi', '9000000004', 'west@lib.com');

INSERT INTO Publisher VALUES

- (1, 'Pearson', 'Delhi', '9100000001'),
- (2, 'Springer', 'Mumbai', '9100000002'),
- (3, 'McGraw Hill', 'Bangalore', '9100000003'),
- (4, 'Oxford Press', 'Chennai', '9100000004');

INSERT INTO Book VALUES

('ISBN001', 'Database Systems', 2020, 1),
('ISBN002', 'Operating Systems', 2019, 2),
('ISBN003', 'Computer Networks', 2021, 3),
('ISBN004', 'Machine Learning', 2022, 4),
('ISBN005', 'Data Structures', 2018, 1);

INSERT INTO Author VALUES

(1, 'Korth', '1960-01-10', 'American'),
(2, 'Silberschatz', '1952-05-11', 'American'),
(3, 'Tanenbaum', '1950-03-16', 'Dutch'),
(4, 'Andrew Ng', '1976-04-18', 'American'),
(5, 'Mark Allen', '1980-07-20', 'British');

INSERT INTO Book_Author VALUES

('ISBN001', 1),
('ISBN002', 2),
('ISBN003', 3),
('ISBN004', 4),
('ISBN005', 5);

INSERT INTO Employee VALUES

(101, 'Riya', '8000000001', 'Delhi', 'Manager', 1),
(102, 'Aman', '8000000002', 'Delhi', 'Assistant', 2),
(103, 'Sneha', '8000000003', 'Delhi', 'Manager', 3),
(104, 'Karan', '8000000004', 'Delhi', 'Assistant', 4),
(105, 'Priya', '8000000005', 'Delhi', 'Librarian', 1);

INSERT INTO Member VALUES

(201, 'Rahul', '7000000001', 'rahul@mail.com', 'Delhi', 'Gold', '2024-01-10'),
(202, 'Simran', '7000000002', 'simran@mail.com', 'Delhi', 'Silver', '2024-02-15'),
(203, 'Arjun', '7000000003', 'arjun@mail.com', 'Delhi', 'Gold', '2024-03-01'),
(204, 'Neha', '7000000004', 'neha@mail.com', 'Delhi', 'Silver', '2024-03-10'),
(205, 'Kabir', '7000000005', 'kabir@mail.com', 'Delhi', 'Platinum', '2024-03-15');

INSERT INTO Book_Copy VALUES

(301, 'Available', 1001, 'ISBN001', 1),
(302, 'Issued', 1002, 'ISBN002', 2),
(303, 'Available', 1003, 'ISBN003', 3),
(304, 'Available', 1004, 'ISBN004', 4),
(305, 'Issued', 1005, 'ISBN005', 1),
(306, 'Available', 1006, 'ISBN001', 2),
(307, 'Issued', 1007, 'ISBN003', 3);

```
INSERT INTO Issue_Record VALUES
```

```
(401, '2024-03-01', '2024-03-11', '2024-03-08', 201, 101, 302),
```

```
(402, '2024-03-05', '2024-03-15', '2024-03-12', 202, 102, 301),
```

```
(403, '2024-03-07', '2024-03-17', NULL, 203, 103, 305),
```

```
(404, '2024-03-10', '2024-03-20', NULL, 204, 104, 307),
```

```
(405, '2024-03-12', '2024-03-22', NULL, 205, 105, 302);
```

TABLES:

```
mysql>
mysql>
mysql> SELECT * FROM Library_Branch;
+-----+-----+-----+-----+-----+-----+
| Branch_ID | Branch_Name | Branch_Address | Location | Contact_Number | Email_ID |
+-----+-----+-----+-----+-----+-----+
| 1 | North Branch | Rohini | Delhi | 9000000001 | north@lib.com |
| 2 | East Branch | Laxmi Nagar | Delhi | 9000000002 | east@lib.com |
| 3 | South Branch | Saket | Delhi | 9000000003 | south@lib.com |
| 4 | West Branch | Janakpuri | Delhi | 9000000004 | west@lib.com |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.001 sec)
```

```
mysql> SELECT * FROM Publisher;
+-----+-----+-----+-----+
| Publisher_ID | Publisher_Name | Publisher_Address | Contact_Number |
+-----+-----+-----+-----+
| 1 | Pearson | Delhi | 9100000001 |
| 2 | Springer | Mumbai | 9100000002 |
| 3 | McGraw Hill | Bangalore | 9100000003 |
| 4 | Oxford Press | Chennai | 9100000004 |
+-----+-----+-----+-----+
4 rows in set (0.001 sec)
```

```
mysql> SELECT * FROM Book;
+-----+-----+-----+-----+
| ISBN | Title | Publication_Year | Publisher_ID |
+-----+-----+-----+-----+
| ISBN001 | Database Systems | 2020 | 1 |
| ISBN002 | Operating Systems | 2019 | 2 |
| ISBN003 | Computer Networks | 2021 | 3 |
| ISBN004 | Machine Learning | 2022 | 4 |
| ISBN005 | Data Structures | 2018 | 1 |
+-----+-----+-----+-----+
5 rows in set (0.000 sec)
```

```
mysql> SELECT * FROM Author;
```

Author_ID	Author_Name	Date_of_Birth	Nationality
1	Korth	1960-01-10	American
2	Silberschatz	1952-05-11	American
3	Tanenbaum	1950-03-16	Dutch
4	Andrew Ng	1976-04-18	American
5	Mark Allen	1980-07-20	British

```
5 rows in set (0.000 sec)
```

```
mysql> SELECT * FROM Book_Author;
```

ISBN	Author_ID
ISBN001	1
ISBN002	2
ISBN003	3
ISBN004	4
ISBN005	5

```
5 rows in set (0.000 sec)
```

```
mysql> SELECT * FROM Employee;
```

Employee_ID	Name	Phone_Number	Address	Employee_Type	Branch_ID
101	Riya	8000000001	Delhi	Manager	1
102	Aman	8000000002	Delhi	Assistant	2
103	Sneha	8000000003	Delhi	Manager	3
104	Karan	8000000004	Delhi	Assistant	4
105	Priya	8000000005	Delhi	Librarian	1

```
5 rows in set (0.000 sec)
```

```
mysql> SELECT * FROM Member;
```

Member_ID	Name	Phone_Number	Email_ID	Address	Membership_Type	Membership_Date
201	Rahul	7000000001	rahul@mail.com	Delhi	Gold	2024-01-10
202	Simran	7000000002	simran@mail.com	Delhi	Silver	2024-02-15
203	Arjun	7000000003	arjun@mail.com	Delhi	Gold	2024-03-01
204	Neha	7000000004	neha@mail.com	Delhi	Silver	2024-03-10
205	Kabir	7000000005	kabir@mail.com	Delhi	Platinum	2024-03-15

```
5 rows in set (0.000 sec)
```

```
mysql> SELECT * FROM Book_Copy;
```

Copy_ID	Status	Acquisition_ID	ISBN	Branch_ID
301	Available	1001	ISBN001	1
302	Issued	1002	ISBN002	2
303	Available	1003	ISBN003	3
304	Available	1004	ISBN004	4
305	Issued	1005	ISBN005	1
306	Available	1006	ISBN001	2
307	Issued	1007	ISBN003	3

```
7 rows in set (0.000 sec)
```

```
mysql> SELECT * FROM Issue_Record;
```

Issue_ID	Issue_Date	Due_Date	Return_Date	Member_ID	Employee_ID	Copy_ID
401	2024-03-01	2024-03-11	2024-03-08	201	101	302
402	2024-03-05	2024-03-15	2024-03-12	202	102	301
403	2024-03-07	2024-03-17	NULL	203	103	305
404	2024-03-10	2024-03-20	NULL	204	104	307
405	2024-03-12	2024-03-22	NULL	205	105	302

```
5 rows in set (0.001 sec)
```

Queries :

Q1) Show Issued Books with Member Name and Issue Date

```
SELECT b.Title, m.Name, i.Issue_Date
```

```
FROM Issue_Record i
```

```
JOIN Member m ON i.Member_ID = m.Member_ID
```

```
JOIN Book_Copy bc ON i.Copy_ID = bc.Copy_ID
```

```
JOIN Book b ON bc.ISBN = b.ISBN
```

```
WHERE bc.Status = 'Issued';
```

```
[mysql>
mysql> SELECT b.Title, m.Name, i.Issue_Date
      -> FROM Issue_Record i
      -> JOIN Member m ON i.Member_ID = m.Member_ID
      -> JOIN Book_Copy bc ON i.Copy_ID = bc.Copy_ID
      -> JOIN Book b ON bc.ISBN = b.ISBN
      -> WHERE bc.Status = 'Issued';
```

Title	Name	Issue_Date
Operating Systems	Rahul	2024-03-01
Operating Systems	Kabir	2024-03-12
Data Structures	Arjun	2024-03-07
Computer Networks	Neha	2024-03-10

```
4 rows in set (0.002 sec)
```

```
[mysql>
```

Q2) Show Available Books with Branch Name

SELECT lb.Branch_Name, b.Title

FROM Book_Copy bc

JOIN Library_Branch lb ON bc.Branch_ID = lb.Branch_ID

JOIN Book b ON bc.ISBN = b.ISBN

WHERE bc.Status = 'Available';

```
[mysql>
[mysql>
mysql> SELECT lb.Branch_Name, b.Title
      -> FROM Book_Copy bc
      -> JOIN Library_Branch lb ON bc.Branch_ID = lb.Branch_ID
      -> JOIN Book b ON bc.ISBN = b.ISBN
      -> WHERE bc.Status = 'Available';
+-----+-----+
| Branch_Name | Title                |
+-----+-----+
| North Branch | Database Systems    |
| South Branch | Computer Networks   |
| West Branch  | Machine Learning    |
| East Branch  | Database Systems    |
+-----+-----+
4 rows in set (0.001 sec)
```

Q3) Show Members Who Have Issued More Than One Book

```
SELECT m.Name, COUNT(i.Issue_ID) AS Books_Issued
FROM Member m
JOIN Issue_Record i ON m.Member_ID = i.Member_ID
GROUP BY m.Name
HAVING COUNT(i.Issue_ID) > 1;
```

```
[mysql>
[mysql>
mysql> SELECT m.Name, COUNT(i.Issue_ID) AS Books_Issued
      -> FROM Member m
      -> JOIN Issue_Record i ON m.Member_ID = i.Member_ID
      -> GROUP BY m.Name
      -> HAVING COUNT(i.Issue_ID) > 1;
Empty set (0.004 sec)
```

Q4) Show Number of Authors for Each Book

```
SELECT b.Title, COUNT(ba.Author_ID) AS Author_Count
```

```
FROM Book b
```

```
JOIN Book_Author ba ON b.ISBN = ba.ISBN
```

```
GROUP BY b.Title;
```

```
[mysql>
mysql> SELECT b.Title, COUNT(ba.Author_ID) AS Author_Count
      -> FROM Book b
      -> JOIN Book_Author ba ON b.ISBN = ba.ISBN
      -> GROUP BY b.Title;
```

Title	Author_Count
Database Systems	1
Operating Systems	1
Computer Networks	1
Machine Learning	1
Data Structures	1

```
5 rows in set (0.001 sec)
```

Q5) Find All Books with Publisher Name

SELECT b.Title, p.Publisher_Name

FROM Book b

JOIN Publisher p ON b.Publisher_ID = p.Publisher_ID;

```
mysql>
[mysql>
mysql> SELECT b.Title, p.Publisher_Name
      -> FROM Book b
      -> JOIN Publisher p ON b.Publisher_ID = p.Publisher_ID;
+-----+-----+
| Title           | Publisher_Name |
+-----+-----+
| Database Systems | Pearson        |
| Data Structures  | Pearson        |
| Operating Systems | Springer       |
| Computer Networks | McGraw Hill    |
| Machine Learning | Oxford Press   |
+-----+-----+
5 rows in set (0.001 sec)
```

Q6) Show Issued Books with Member Name

```
SELECT m.Name, b.Title, i.Issue_Date
```

```
FROM Issue_Record i
```

```
JOIN Member m ON i.Member_ID = m.Member_ID
```

```
JOIN Book_Copy bc ON i.Copy_ID = bc.Copy_ID
```

```
JOIN Book b ON bc.ISBN = b.ISBN;
```

```
mysql>
mysql> SELECT m.Name, b.Title, i.Issue_Date
-> FROM Issue_Record i
-> JOIN Member m ON i.Member_ID = m.Member_ID
-> JOIN Book_Copy bc ON i.Copy_ID = bc.Copy_ID
-> JOIN Book b ON bc.ISBN = b.ISBN;
```

Name	Title	Issue_Date
Rahul	Operating Systems	2024-03-01
Simran	Database Systems	2024-03-05
Arjun	Data Structures	2024-03-07
Neha	Computer Networks	2024-03-10
Kabir	Operating Systems	2024-03-12

```
5 rows in set (0.001 sec)
```

Q7) Count Books in Each Branch

```
SELECT lb.Branch_Name, COUNT(bc.Copy_ID) AS Total_Books
FROM Library_Branch lb
JOIN Book_Copy bc ON lb.Branch_ID = bc.Branch_ID
GROUP BY lb.Branch_Name;
```

```
[mysql>
mysql> SELECT lb.Branch_Name, COUNT(bc.Copy_ID) AS Total_Books
      -> FROM Library_Branch lb
      -> JOIN Book_Copy bc ON lb.Branch_ID = bc.Branch_ID
      -> GROUP BY lb.Branch_Name;
+-----+-----+
| Branch_Name | Total_Books |
+-----+-----+
| North Branch |          2 |
| East Branch  |          2 |
| South Branch |          2 |
| West Branch  |          1 |
+-----+-----+
4 rows in set (0.002 sec)
```

Q8) Members Who Have Not Returned Books

```
SELECT m.Name, b.Title
FROM Issue_Record i
JOIN Member m ON i.Member_ID = m.Member_ID
JOIN Book_Copy bc ON i.Copy_ID = bc.Copy_ID
JOIN Book b ON bc.ISBN = b.ISBN
WHERE i.Return_Date IS NULL;
```

```
[mysql>
[mysql>
mysql> SELECT m.Name, b.Title
      -> FROM Issue_Record i
      -> JOIN Member m ON i.Member_ID = m.Member_ID
      -> JOIN Book_Copy bc ON i.Copy_ID = bc.Copy_ID
      -> JOIN Book b ON bc.ISBN = b.ISBN
      -> WHERE i.Return_Date IS NULL;
+-----+-----+
| Name  | Title                |
+-----+-----+
| Arjun | Data Structures      |
| Neha  | Computer Networks    |
| Kabir | Operating Systems    |
+-----+-----+
3 rows in set (0.000 sec)
```

Q9) Books with Multiple Authors

```
SELECT b.Title, COUNT(ba.Author_ID) AS Authors
FROM Book b
JOIN Book_Author ba ON b.ISBN = ba.ISBN
GROUP BY b.Title
HAVING COUNT(ba.Author_ID) > 1;
```

```
[mysql>
mysql> SELECT b.Title, COUNT(ba.Author_ID) AS Authors
      -> FROM Book b
      -> JOIN Book_Author ba ON b.ISBN = ba.ISBN
      -> GROUP BY b.Title
      -> HAVING COUNT(ba.Author_ID) > 1;
Empty set (0.002 sec)
```

Q10) Total Books Issued by Each Member

```
SELECT m.Name, COUNT(i.Issue_ID) AS Total_Issued  
FROM Member m  
JOIN Issue_Record i ON m.Member_ID = i.Member_ID  
GROUP BY m.Name;
```

```
mysql>  
mysql> SELECT m.Name, COUNT(i.Issue_ID) AS Total_Issued  
-> FROM Member m  
-> JOIN Issue_Record i ON m.Member_ID = i.Member_ID  
-> GROUP BY m.Name;  
  
+-----+-----+  
| Name   | Total_Issued |  
+-----+-----+  
| Rahul  | 1            |  
| Simran | 1            |  
| Arjun  | 1            |  
| Neha   | 1            |  
| Kabir  | 1            |  
+-----+-----+  
5 rows in set (0.001 sec)
```

Q11) Branch-wise Available Books

```
SELECT lb.Branch_Name, b.Title  
FROM Book_Copy bc  
JOIN Library_Branch lb ON bc.Branch_ID = lb.Branch_ID  
JOIN Book b ON bc.ISBN = b.ISBN  
WHERE bc.Status = 'Available';
```

```
[mysql>  
[mysql>  
mysql> SELECT lb.Branch_Name, b.Title  
-> FROM Book_Copy bc  
-> JOIN Library_Branch lb ON bc.Branch_ID = lb.Branch_ID  
-> JOIN Book b ON bc.ISBN = b.ISBN  
-> WHERE bc.Status = 'Available';  
+-----+-----+  
| Branch_Name | Title |  
+-----+-----+  
| North Branch | Database Systems |  
| South Branch | Computer Networks |  
| West Branch | Machine Learning |  
| East Branch | Database Systems |  
+-----+-----+  
4 rows in set (0.001 sec)
```

Normalization

1. Introduction to Normalization

Normalization is the systematic process of organizing a relational database to reduce data redundancy and improve data integrity. It works by decomposing tables into smaller, well-structured relations based on a set of formal rules called Normal Forms (NF). Each normal form imposes progressively stricter constraints on the schema to eliminate specific types of anomalies:

- Insertion Anomaly: Inability to add data without including unrelated data.
- Deletion Anomaly: Loss of important data when deleting a record.
- Update Anomaly: Inconsistency arising when the same data must be updated in multiple places.

The Multi-Branch Library Management System database is analyzed through each normalization stage: $UNF \rightarrow 1NF \rightarrow 2NF \rightarrow 3NF \rightarrow BCNF$. The goal is to confirm that the final schema is free from all such anomalies and follows best practices in relational database design.

2. Unnormalized Form (UNF)

In the Unnormalized Form, data is stored without any structural constraints. A single conceptual flat record for the library system would contain repeating groups and multi-valued attributes. For example, a single row might look like:

```
LIBRARY_FLAT (  
  Branch_ID, Branch_Name, Branch_Address, Location, Contact_Number,  
  Employee_ID, Employee_Name, Employee_Type,  
  ISBN, Title, Publication_Year,  
  Author_ID_1, Author_Name_1, Author_ID_2, Author_Name_2, ...   (Repeating  
Group)  
  Publisher_ID, Publisher_Name, Publisher_Address,  
  Copy_ID, Status, Acquisition_ID,  
  Member_ID, Member_Name, Membership_Type,  
  Issue_ID, Issue_Date, Due_Date, Return_Date  
)
```

Problems in UNF:

- Repeating group for Author: a book can have multiple authors, making the structure non-atomic.
- Massive redundancy: branch and publisher details repeat for every book copy issued.
- Deletion anomaly: deleting a member record may lose issue transaction history.
- Insert anomaly: a new branch cannot be recorded until at least one employee is assigned.

3. First Normal Form (1NF)

Rules for 1NF:

- All attributes must contain only atomic (indivisible) values.
- No repeating groups or arrays are allowed.
- Each row must be uniquely identifiable by a primary key.

The repeating Author group is eliminated by introducing a separate Book_Author junction table. Every attribute in every table now holds a single atomic value and each table has a clearly defined primary key.

Schema after 1NF:

```
Library_Branch  ( Branch_ID*, Branch_Name, Branch_Address, Location,
                  Contact_Number, Email_ID )

Publisher        ( Publisher_ID*, Publisher_Name, Publisher_Address,
                  Contact_Number )

Book             ( ISBN*, Title, Publication_Year, Publisher_ID )

Author          ( Author_ID*, Author_Name, Date_of_Birth, Nationality )

Book_Author     ( ISBN*, Author_ID* )          -- Junction table; eliminates
repeating group

Employee        ( Employee_ID*, Name, Phone_Number, Address,
                  Employee_Type, Branch_ID )

Member          ( Member_ID*, Name, Phone_Number, Email_ID,
                  Address, Membership_Type, Membership_Date )

Book_Copy       ( Copy_ID*, Status, Acquisition_ID, ISBN, Branch_ID )

Issue_Record    ( Issue_ID*, Issue_Date, Due_Date, Return_Date,
                  Member_ID, Employee_ID, Copy_ID )
```

All tables now satisfy 1NF. Every attribute is atomic and every table has a well-defined primary key (marked with *).

4. Second Normal Form (2NF)

Rules for 2NF:

- The relation must already be in 1NF.
- Every non-key attribute must be fully functionally dependent on the entire primary key.
- Partial dependency (a non-key attribute depending on only part of a composite key) must be eliminated.

Partial dependency can only occur in tables with a composite primary key. The only such table in the schema is Book_Author.

Analysis:

Book_Author (ISBN, Author_ID) -- Composite Primary Key

There are no non-key attributes in this table.
Therefore, no partial dependency exists.
Result: Already in 2NF.

All remaining tables (Library_Branch, Publisher, Book, Author, Employee, Member, Book_Copy, Issue_Record) have single-attribute primary keys. Partial dependency is impossible in single-key tables.

Conclusion: The entire database schema is already in 2NF. No decomposition is required.

5. Third Normal Form (3NF)

Rules for 3NF:

- The relation must already be in 2NF.
- No transitive dependency must exist, i.e., no non-key attribute should depend on another non-key attribute.

Analysis for Transitive Dependencies:

Book (ISBN, Title, Publication_Year, Publisher_ID)

ISBN -> Publisher_ID (direct FK dependency, not transitive).
Publisher details are stored in a separate Publisher table.
Result: No transitive dependency. Already in 3NF.

Employee (Employee_ID, Name, Phone_Number, Address, Employee_Type, Branch_ID)

All non-key attributes depend directly on Employee_ID.
Branch details are stored in Library_Branch, not repeated here.
Result: No transitive dependency. Already in 3NF.

Member (Member_ID, Name, Phone_Number, Email_ID, Address, Membership_Type, Membership_Date)

All attributes depend solely and directly on Member_ID.

Result: No transitive dependency. Already in 3NF.

Issue_Record (Issue_ID, Issue_Date, Due_Date, Return_Date, Member_ID, Employee_ID, Copy_ID)

No non-key attribute depends on another non-key attribute.

Member, Employee, and Copy_ID are foreign keys pointing to their own tables.

Result: No transitive dependency. Already in 3NF.

Conclusion: The complete database schema satisfies 3NF. No further decomposition is needed.

6. Boyce-Codd Normal Form (BCNF)

Rules for BCNF:

- The relation must already be in 3NF.
- For every non-trivial functional dependency $X \rightarrow Y$, X must be a superkey.

BCNF is a stricter version of 3NF. It closes the gap where 3NF may still allow anomalies in certain edge cases involving candidate keys.

Analysis:

Book_Author (ISBN, Author_ID) -- Composite PK: (ISBN, Author_ID)

Only FD: (ISBN, Author_ID) $\rightarrow$ (no additional attributes).

The determinant is the full composite key, which is a superkey.

Result: Satisfies BCNF.

All single-PK tables (Library_Branch, Book, Author, Publisher, Employee, Member, Book_Copy, Issue_Record):

Each table has a single primary key.

All functional dependencies have the primary key as their determinant.

The primary key is always a superkey by definition.

Result: All tables satisfy BCNF.

Conclusion: The complete Multi-Branch Library Management System schema satisfies BCNF. No decomposition is required at this stage.

Member, Employee, and Copy_ID are foreign keys pointing to their own tables.

Result: No transitive dependency. Already in 3NF.

Conclusion: The complete database schema satisfies 3NF. No further decomposition is needed.

7. Functional Dependencies (FDs)

The following functional dependencies are identified across all tables in the final schema. Attributes forming the primary key are marked with *.

Library_Branch :

Branch_ID* -> Branch_Name, Branch_Address, Location,
Contact_Number, Email_ID

Publisher :

Publisher_ID* -> Publisher_Name, Publisher_Address, Contact_Number

Book :

ISBN* -> Title, Publication_Year, Publisher_ID

Author :

Author_ID* -> Author_Name, Date_of_Birth, Nationality

Book_Author :

(ISBN, Author_ID)* -> (no additional attributes)

Employee :

Employee_ID* -> Name, Phone_Number, Address, Employee_Type, Branch_ID

Member :

Member_ID* -> Name, Phone_Number, Email_ID, Address,
Membership_Type, Membership_Date

Book_Copy :

Copy_ID* -> Status, Acquisition_ID, ISBN, Branch_ID

Issue_Record :

Issue_ID* -> Issue_Date, Due_Date, Return_Date,
Member_ID, Employee_ID, Copy_ID

8. Final Normalized Schema

After applying all normalization rules (1NF through BCNF), the final relational schema is presented below. Primary Keys are marked with * and Foreign Keys are noted with (FK).

Library_Branch (Branch_ID*, Branch_Name, Branch_Address,
Location, Contact_Number, Email_ID)

Publisher (Publisher_ID*, Publisher_Name, Publisher_Address,
Contact_Number)

Book (ISBN*, Title, Publication_Year,
Publisher_ID (FK -> Publisher))

Author (Author_ID*, Author_Name, Date_of_Birth, Nationality)
 Book_Author (ISBN* (FK -> Book), Author_ID* (FK -> Author))
 Composite PK: (ISBN, Author_ID)
 Employee (Employee_ID*, Name, Phone_Number, Address,
 Employee_Type, Branch_ID (FK -> Library_Branch))
 Member (Member_ID*, Name, Phone_Number, Email_ID,
 Address, Membership_Type, Membership_Date)
 Book_Copy (Copy_ID*, Status, Acquisition_ID,
 ISBN (FK -> Book), Branch_ID (FK -> Library_Branch))
 Issue_Record (Issue_ID*, Issue_Date, Due_Date, Return_Date,
 Member_ID (FK -> Member),
 Employee_ID (FK -> Employee),
 Copy_ID (FK -> Book_Copy))

Normalization Summary :

Table	1NF	2NF	3NF	BCNF	Notes
Library_Branch	Yes	Yes	Yes	Yes	Single PK
Publisher	Yes	Yes	Yes	Yes	Single PK
Book	Yes	Yes	Yes	Yes	No transitive deps
Author	Yes	Yes	Yes	Yes	Single PK
Book_Author	Yes	Yes	Yes	Yes	Composite PK, no extra attrs
Employee	Yes	Yes	Yes	Yes	Single PK
Member	Yes	Yes	Yes	Yes	Single PK
Book_Copy	Yes	Yes	Yes	Yes	Single PK
Issue_Record	Yes	Yes	Yes	Yes	Ternary relationship

Conclusion

The Multi-Branch Library Management System database has been rigorously analyzed through all stages of normalization from UNF to BCNF. The following points summarize the findings:

- 1NF: Repeating groups eliminated. The Book_Author junction table resolves the many-to-many Author relationship, making all attributes atomic. Each table has a well-defined primary key.
- 2NF: No partial dependencies found. The only composite-key table (Book_Author) contains no non-key attributes, so no partial dependency can exist. All other tables use single primary keys.
- 3NF: No transitive dependencies found. Every non-key attribute in every table depends directly and solely on the primary key of that table. Related details (publisher, branch, member, employee) are always stored in their own dedicated tables.
- BCNF: Every determinant in every functional dependency across all tables is a superkey. The schema fully satisfies BCNF without requiring any further decomposition.

The final normalized schema is clean, efficient, and free from insertion, deletion, and update anomalies. It accurately represents the real-world library domain while maintaining referential integrity through well-defined primary and foreign key constraints. This normalized design provides a solid and reliable foundation for all the SQL queries implemented in this lab file

